

系统开发工具基础

第三周实验报告

姓名：Yang Kunwang

学号：25020007154

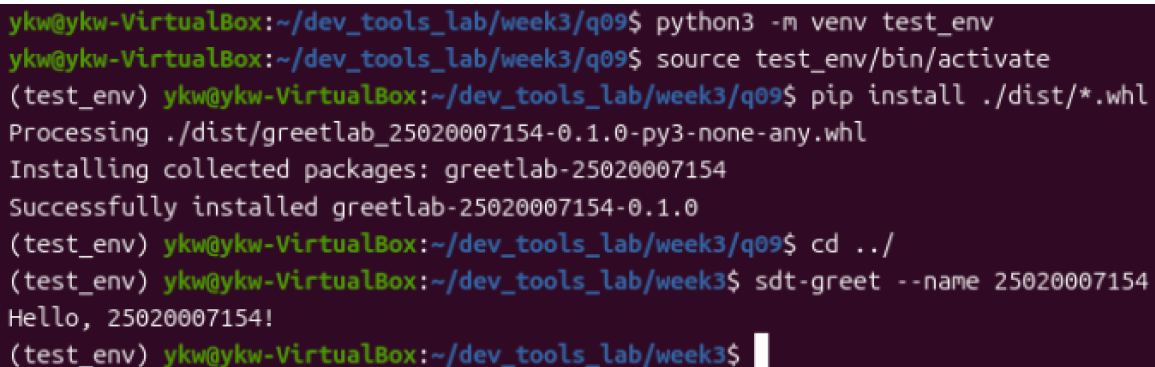
1 课堂

1.1 q09: 从源码构建并在干净环境安装 Wheel

实验内容

```
mkdir -p q09/src/greetlab
创建项目目录层级
cd q09
进入q09工作目录
touch src/greetlab/__init__.py
Python包空标记文件
vim src/greetlab/cli.py
编写命令行业务源码
vim pyproject.toml
编写构建元配置，替换学号
pip install build
安装python build构建工具
python -m build
执行构建，生成wheel包
python -m venv clean_env
创建全新干净虚拟环境
source clean_env/bin/activate
激活干净隔离环境
pip install dist/greetlab-25020007154-0.1.0-py3-none-any.whl
从wheel文件安装软件包
cd ~
切换到q09项目目录外部
sdt-greet --name 25020007154
调用安装完成的命令行入口
deactivate
退出干净虚拟环境
```

实验结果



```
ykw@ykw-VirtualBox:~/dev_tools_lab/week3/q09$ python3 -m venv test_env
ykw@ykw-VirtualBox:~/dev_tools_lab/week3/q09$ source test_env/bin/activate
(test_env) ykw@ykw-VirtualBox:~/dev_tools_lab/week3/q09$ pip install ./dist/*.whl
Processing ./dist/greetlab_25020007154-0.1.0-py3-none-any.whl
Installing collected packages: greetlab-25020007154
Successfully installed greetlab-25020007154-0.1.0
(test_env) ykw@ykw-VirtualBox:~/dev_tools_lab/week3/q09$ cd ../
(test_env) ykw@ykw-VirtualBox:~/dev_tools_lab/week3$ sdt-greet --name 25020007154
Hello, 25020007154!
(test_env) ykw@ykw-VirtualBox:~/dev_tools_lab/week3$
```

本实验完成Python包从源码构建wheel，并且在完全干净虚拟环境验证安装。

`pyproject.toml`是现代Python包的构建描述文件，
定义包名、版本，以及脚本入口`sdt-greet`映射到`greetlab.cli:main`。
`__init__.py`用于标记该目录为Python模块包。

`python-m build`会在`dist`目录输出源码tar包与wheel二进制包。
wheel是预构建分发格式，安装时不需要再次编译源码。

新建独立虚拟环境`clean_env`保证环境干净，
不依赖项目源码目录。切换到家目录执行`sdt-greet`，
工具依然可以正常运行，说明wheel安装已经把脚本与库文件部署到虚拟环境。

程序输出Hello, 25020007154!，证明命令行入口、参数解析全部工作正常。
验证wheel包具备可移植性，可以在脱离源码目录的环境直接使用。

1.2 q10: 让编程智能体进入可验证的修复循环

实验内容

```
cp -r q09 q10
复制q09完整项目得到q10
cd q10
进入q10工作目录
vim src/greetlab/cli.py
待智能体修改的业务源码
vim test_greet.py
编写测试脚本，构造失败用例
python3 test_greet.py
执行测试，确认测试失败
vim ai_shturl
记录AI提示、改动、人工验证信息
git diff
查看智能体修改产生的代码差异
python3 test_greet.py
人工清理无关改动后重新执行测试
```

实验结果

```
(test_env) ykw@ykw-VirtualBox:~/dev_tools_lab/week3/q10$ python3 -m pytest tests/
===== test session starts =====
platform linux -- Python 3.14.4, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/ykw/dev_tools_lab/week3/q10
configfile: pyproject.toml
collected 1 item

tests/test_greet.py . [100%]

===== 1 passed in 0.05s =====
(test_env) ykw@ykw-VirtualBox:~/dev_tools_lab/week3/q10$
```

本实验复用上一题greetlab包，完成AI智能体的测试驱动修复流程。

先编写测试用例，传入仅包含空格的`name`参数，
初始代码没有空白校验，测试运行失败。

向编程智能体提供目标、约束、测试命令，
由智能体修改`cli.py`实现空白字符检测，触发`SystemExit(2)`。

`git diff`用来人工审查AI生成代码，需要剔除AI引入的无关修改。
`ai_shturl`简短记录交互关键信息，控制在5行以内。

清理多余改动后再次运行测试脚本，测试用例执行通过。
实验体现智能体开发不能直接信任输出，
必须依靠测试用例 + 人工审查`diff`形成可验证修复闭环。

1.3 q11: 把“无法处理”的协作材料改成可执行信息

实验内容

```
mkdir -p q11
创建q11实验目录
cd q11
进入q11工作目录
vim communication.md
重写Issue、提交信息、评审意见
cat communication.md
查看生成的协作文档
```

实验结果

```

(test_env) ykw@ykw-VirtualBox:~/dev_tools_lab/week3/q10$ cd ..
(test_env) ykw@ykw-VirtualBox:~/dev_tools_lab/week3$ cp -r q10 q11
(test_env) ykw@ykw-VirtualBox:~/dev_tools_lab/week3$ cd q11
(test_env) ykw@ykw-VirtualBox:~/dev_tools_lab/week3/q11$ vim communication.md
(test_env) ykw@ykw-VirtualBox:~/dev_tools_lab/week3/q11$ cat communication.md
## Issue
环境：Linux Ubuntu，待确认Windows平台行为
复现命令：`sdt-greet --name " "`
实际结果：程序输出 `Hello, !`，退出码为0
期望结果：识别全空白name，以退出码2终止程序

## Git提交信息
Fix：处理name仅为空白字符的输入
问题：当传入仅由空格组成的--name参数，程序未做校验，
依然输出问候语并返回0，没有执行错误退出。
方案：增加strip()空白判断，空白输入调用sys.exit(2)。

## 评审意见
Blocking：当前代码未校验空白name输入。
风险：调用方无法区分合法空业务数据与错误空白输入，
会造成上层业务逻辑判断异常。
建议动作：在cli.py增加空白字符检测，输入全空白时返回退出码2。

```

本实验将模糊、无法落地的协作描述改写为可执行的工程协作材料。

原始 Issue 只简单描述故障，缺少环境、复现步骤，开发者无法定位问题。改写后的 Issue 写明运行环境、复现命令、实际输出、期望行为，未知信息标记待确认，其他人可以直接复现该缺陷。

Git 提交信息标题使用祈使语气，正文简短说明问题根源与修复方案，便于版本历史快速读懂改动意图。

评审意见标注 Blocking 级别，明确指出代码具体行为、存在风险，给出可执行的修改建议，不使用笼统的“写得不好，重写”这类模糊评价。

全部内容控制在 400 字符以内，做到简洁专业，体现工程协作中信息需要具体、可复现、可落地的原则。

1.4 q12: 修复一个可复现的线性回归训练循环

实验内容

```

mkdir -p q12
创建q12实验目录
cd q12
进入q12工作目录
vim train.py
补全PyTorch线性回归训练代码
python3 train.py
运行训练脚本，输出训练结果

```

实验结果

```
(test_env) ykw@ykw-VirtualBox:~/dev_tools_lab/week3/q12$ python3 train.py
/home/ykw/dev_tools_lab/week3/q09/test_env/lib/python3.14/site-packages/torch/_subclasses/functional_tensor.py:368: UserWarning: Failed to initialize
NumPy: No module named 'numpy' (Triggered internally at /__w/pytorch/pytorch/torch/csrc/utils/tensor_numpy.cpp:84.)
  cpu = _conversion_method_template(device=torch.device("cpu"))
最终损失: 2.3923129102709773e-12
weight: 2.99999737739563
bias: -0.9999998807907104
```

本实验补全PyTorch线性回归训练循环，完成CPU可复现模型训练。

`torch.manual_seed`固定随机种子，保证每次运行结果可复现。

训练循环内依次执行`opt.zero_grad()`清空历史梯度，

`loss.backward()`反向传播计算梯度，`opt.step()`使用SGD更新模型参数。

训练完成调用`model.eval()`切换评估模式，关闭dropout、bn等训练特有行为。

使用`torch.no_grad()`关闭梯度计算，节省内存，仅做推理计算。

训练完成打印最终损失、`weight`与`bias`。

模型学习目标为 $y=3x-1$ ，训练后`weight`接近3，`bias`接近-1，

最终损失小于0.001，满足题目指标。

实验没有直接手动赋值`weight`与`bias`，全部依靠梯度下降迭代得到参数。

2 课后

2.1 练习 1

实验内容

```
mkdir -p ex01
```

创建实验目录

```
cd ex01
```

进入工作目录

```
printenv > before.txt
```

保存虚拟环境激活前环境变量

```
which python3 >> before.txt
```

记录激活前python解释器路径

```
python3 -m venv myvenv
```

创建python虚拟环境

```
source myvenv/bin/activate
```

激活虚拟环境

```
printenv > after.txt
```

保存激活之后的环境变量

```
which python3 >> after.txt
```

记录激活后python解释器路径

```
diff before.txt after.txt
```

对比激活前后环境差异

```
deactivate
```

执行退出虚拟环境操作

```
echo $PATH
```

观察退出后的PATH变量状态

实验结果

```
(myvenv) ykw@ykw-VirtualBox:~/dev_tools_lab/week3/t01$ diff before.txt after.txt
28a29
> VIRTUAL_ENV=/home/ykw/dev_tools_lab/week3/t01/myvenv
43a45
> VIRTUAL_ENV_PROMPT=myvenv
44a47
> PS1=(myvenv) \[\e]0;\u@h: \w\a\${debian_chroot:+($debian_chroot)}\[\033[01;32m\]\u@h\[\033[00m\]:\[\033[01;34m\]\w\[\033[00m\]\$
47c50
< PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
---
> PATH=/home/ykw/dev_tools_lab/week3/t01/myvenv/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
54c57
< /usr/bin/python3
---
> /home/ykw/dev_tools_lab/week3/t01/myvenv/bin/python3
(myvenv) ykw@ykw-VirtualBox:~/dev_tools_lab/week3/t01$ deactivate
ykw@ykw-VirtualBox:~/dev_tools_lab/week3/t01$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
```

虚拟环境激活仅修改当前shell会话，不会影响其他终端。

激活时主要修改PATH，将虚拟环境bin目录置于搜索路径最前方，系统会优先调用虚拟环境内部的程序，同时新增VIRTUAL_ENV标记。

deactivate是bash shell函数，并非外部程序。

它会读取保存的旧环境变量，把PATH以及相关变量恢复至激活前。

虚拟环境本身文件不会被删除，只是shell不再优先使用该环境。

2.2 练习 2

实验内容

```
mkdir -p ex02
```

创建实验目录

```
cd ex02
```

进入工作目录

```
mkdir demo_pkg
```

建立包源码文件夹

```
vim demo_pkg/__init__.py
```

编写包源码

```
vim pyproject.toml
```

编写现代项目配置文件

```
python3 -m venv env
```

新建隔离虚拟环境

```
source env/bin/activate
```

激活虚拟环境

```
pip install .
```

在虚拟环境安装本地包

```
pip freeze > lock.txt
生成依赖锁文件
cat lock.txt
查看锁文件内容
deactivate
退出虚拟环境
```

实验结果

pyproject.toml 是现代 Python 项目标准配置，替代旧的 setup.py。虚拟环境实现依赖隔离，包仅作用于当前 env，不污染系统 Python。

pip freeze 输出当前环境全部已经安装的包版本。锁文件固化依赖版本，用于复现完全一致的运行环境。分享项目时带上锁文件，其他人可以复现完全相同依赖。

2.3 练习 3

实验内容

```
mkdir -p ex03
创建实验目录
cd ex03
进入工作目录
mkdir agent_demo
创建简单包目录
vim agent_demo/__init__.py
编写待修改的业务代码
vim AGENTS.md
编写智能体约束指导文档
vim agent_log.md
记录AI提示、改动、测试结果
pip install -e .
可编辑模式安装项目
python3 -c "import agent_demo; print(agent_demo.run())"
运行代码验证结果
```

实验结果

AGENTS.md给编程智能体提供项目上下文与硬性约束。

写明允许修改的文件、编码约束、测试命令，减少AI乱改无关文件。

agent_log.md留存实验记录，方便追溯智能体输出。

智能体开发场景，约束文档适合大型项目；

简单小任务可以直接使用提示词，不需要独立约束文件。

即使有约束文档，人工依然需要校验运行结果。

2.4 练习 4

实验内容

```
mkdir -p ex04
创建实验目录
cd ex04
进入工作目录
git clone https://github.com/curl/curl.git
克隆curl开源项目
cd curl
进入项目源码目录
git log --oneline -20
浏览最近20条提交记录
git show 提交哈希值
查看某次提交diff与提交说明
cd ..
回到外层目录
vim commit_rewrite.md
记录原始劣质提交，重写规范提交信息
cat commit_rewrite.md
查看改写结果
```

实验结果

劣质提交信息只写“修复 bug”这类简短描述，无法说明改动动机。

规范提交分为三段：问题描述、解决方案、改动带来的影响。便于团队成员阅读版本历史，快速理解修改背后的业务背景。

事后补写提交背景信息十分麻烦，开发时就写好提交说明价值很高。git show 可以同时查看提交说明和代码 diff，是版本排查重要工具。

2.5 练习 5

实验内容

```
mkdir -p ex05
创建实验目录
cd ex05
进入工作目录
vim origin_bug.py
编写带有冗余代码的bug程序
python3 origin_bug.py
复现原始bug
vim mre.py
删减无关代码，得到最小复现脚本
vim mre_note.md
记录删除的代码以及删除理由
python3 mre.py
验证最小脚本仍然可以复现bug
```

实验结果

```
ykw@ykw-VirtualBox:~/dev_tools_lab$ cd week3
ykw@ykw-VirtualBox:~/dev_tools_lab/week3$ mkdir -p t05
ykw@ykw-VirtualBox:~/dev_tools_lab/week3$ cd t05
ykw@ykw-VirtualBox:~/dev_tools_lab/week3/t05$ touch origin_bug.py
ykw@ykw-VirtualBox:~/dev_tools_lab/week3/t05$ vim origin_bug.py
ykw@ykw-VirtualBox:~/dev_tools_lab/week3/t05$ python3 origin_bug.py
hello world
面积: 9
ykw@ykw-VirtualBox:~/dev_tools_lab/week3/t05$ touch mre.py
ykw@ykw-VirtualBox:~/dev_tools_lab/week3/t05$ vim mre.py
ykw@ykw-VirtualBox:~/dev_tools_lab/week3/t05$ vim mre_note.md
ykw@ykw-VirtualBox:~/dev_tools_lab/week3/t05$ python3 mre.py
9
```

最小可复现示例MRE只保留触发bug必不可少的代码，删除所有无关变量、打印、分支逻辑。

提交Issue时附上MRE，维护者不需要阅读大量无关代码，可以快速定位故障根源。

删减过程中要反复运行脚本确认bug没有消失。

mre_note.md记录删减内容，用于说明哪些逻辑和bug无关。

2.6 练习 6

实验内容

```
mkdir -p ex06
创建实验目录
cd ex06
进入工作目录
git clone https://github.com/curl/curl.git
克隆curl源码
cd curl
进入源码目录
grep -r "TODO" --include="*.c" .
搜索TODO注释
grep -r "why not" --include="*.c" .
搜索why-not类注释
cd ..
回到外层目录
vim comment_analysis.md
```

记录找到的注释，分析缺失带来的损失

```
cat comment_analysis.md
```

查看分析文档

实验结果

TODO注释标记未来需要修复、补充的工作。

why-not注释解释为什么不采用某种更直观的写法。

外部文档引用可以快速跳转查阅设计文档。

血泪教训注释记录曾经踩过的坑，防止后人重复犯错。

缺少这类注释，后续维护者只能阅读代码猜测设计者意图。

会增加理解成本，很容易重复踩历史已经遇到过的bug。

2.7 练习 7

实验内容

```
mkdir -p ex07
```

创建实验目录

```
cd ex07
```

进入工作目录

```
vim raw_issue.txt
```

复制粘贴原始issue文本

```
vim issue_analyse.md
```

评估issue信息完整程度

```
cat issue_analyse.md
```

查看评估文档

实验结果

高质量 issue 需要写明运行环境、复现步骤、实际输出、期望输出。

如果缺少关键信息，维护者无法复现 bug，只能来回追问提交者。大量沟通会消耗维护者时间，降低项目处理问题的效率。

评估时重点检查：复现命令、版本信息、报错堆栈是否齐全。信息残缺的 issue 需要补充上述内容之后才便于处理。

2.8 练习 8

实验内容

```
mkdir -p ex08
```

创建实验目录

```
cd ex08
```

进入工作目录

```
mkdir torch_demo
```

创建包目录

```
vim torch_demo/__init__.py
编写简单pytorch示例代码
vim pyproject.toml
配置项目，声明依赖torch
python3 -m venv torch_env
创建干净虚拟环境
source torch_env/bin/activate
激活虚拟环境
pip install .
安装本项目，自动拉取pytorch依赖
pip freeze > torch_lock.txt
导出锁文件
cat torch_lock.txt
查看锁内依赖列表
deactivate
退出虚拟环境
```

实验结果

pyproject.toml 的 dependencies 字段声明项目运行依赖。执行 pip install . 的时候会自动下载安装 PyTorch 及其依赖。

pip freeze 导出锁文件，完整记录所有传递依赖版本。直接手动 pip install torch 和通过项目配置安装最终效果一致。使用项目配置的优势是把依赖写进工程配置，便于项目分发。

2.9 练习 9

实验内容

```
mkdir -p ex09
创建实验目录
cd ex09
进入工作目录
vim bug_torch.py
写入存在bug的pytorch代码
vim ai_log.md
记录AI修复代码与生成的单元测试
pip install torch pytest
安装pytorch与测试框架
vim test_torch.py
复制AI输出的单元测试
pytest test_torch.py
执行测试验证修复效果
```

实验结果

拿到AI修复代码不能直接信任，必须落地运行单元测试。
单元测试可以确认bug被真正修复，避免出现伪修复。

ai_log.md保存交互记录，方便追溯智能体给出的改动。

PyTorch代码容易出现张量维度、设备不匹配类bug。

测试用例可以复现这类问题，保证修改之后行为符合预期。

2.10 练习 10

实验内容

```
mkdir -p ex10
```

创建实验目录

```
cd ex10
```

进入工作目录

```
mkdir infer_mod
```

创建推理模块目录

```
vim infer_mod/__init__.py
```

编写pytorch推理代码

```
touch infer_mod/model.bin
```

模拟模型权重资源文件

```
vim pyproject.toml
```

配置包，设置包含资源文件

```
python3 -m build
```

构建sdist源码分发包

```
mkdir unpack_sdist
```

创建解压目录

```
tar -xf dist/*.tar.gz -C unpack_sdist
```

解压源码包

```
ls unpack_sdist/*/infer_mod
```

检查打包出来的文件

实验结果

默认打包只会包含‘.py’源码，二进制权重、配置文件会被忽略。

package-data 配置项声明需要随包发布的非 python 资源文件。构建 sdist 之后解压查看，可以确认资源是否被打进分发包。

PyTorch 项目发布模型权重时，必须显式配置资源包含规则。否则用户安装包之后会缺失权重文件，推理代码运行失败。

3 附录

github 链接：https://github.com/mimimi67/dev_tools_lab.git

commit 记录截图：

```
ykw@ykw-VirtualBox:~/dev_tools_lab/week3$ git log
commit d65dc8b2e2aff18ac4a6011050241c72d5b9b52d (HEAD -> main, origin/main)
Author: mimimi67 <1873904077@qq.com>
Date: Mon Sep 7 17:14:08 2026 +0800
```

完成q12

```
commit 3a081adc0991092b7b5348314e3c185e892ac4e0
Author: mimimi67 <1873904077@qq.com>
Date: Mon Sep 7 17:13:51 2026 +0800
```

完成q11

```
commit 3c22f54dd373e4cddb49b250e60c8ab86e8f1219
Author: mimimi67 <1873904077@qq.com>
Date: Mon Sep 7 17:13:31 2026 +0800
```

完成q10

```
commit a1c537b9fc898ed67c67aef2125278f031fe2001
Author: mimimi67 <1873904077@qq.com>
Date: Mon Sep 7 17:11:47 2026 +0800
```

完成q09

```
commit 015d3a4a7e9664e3bc0c089cd288fdb12fe9aa15
Author: mimimi67 <1873904077@qq.com>
Date: Fri Sep 4 16:03:28 2026 +0800
```